

Programming Lab 1 (CS21201)
M.Tech. I Semester (2026-27)
Assignment:- 03 (Part: A)

Scheduling Problem

On a computational server, users are submitting jobs to run on a single CPU.

- A user also declares the expected run time of the job.
- Jobs can be preempted.

Policy: Shortest Remaining Processing Time (SRPT), which allows interruption of a job if a new job with a smaller remaining run time is submitted.

The policy minimizes average waiting time.

Required Operations for the Scheduling Problem

- Update the remaining time in every tick.
- Delete a job when the remaining time is zero.
- Find the next job to run.
- Insert a job when it arrives.

Priority Queue

In a priority queue, we dequeue the highest-priority element from the enqueued elements according to their priorities.

```
priority_queue<T, Container, Compare> q;
```

- Allocates a new queue q.
- q.push(e): Adds the given element e to the queue.
- q.pop(): Removes the highest-priority element from the queue.
- q.top(): Accesses the highest-priority element.
- Container class defines the physical data structure where the queue will be stored. The default value is vector.
- Compare class defines the method for comparing the priorities of two elements.

Q1. Implementation of the Scheduling Problem

Give an implementation for the scheduling problem using the C priority_queue.

Priority Queue Using a Linked List

If a linked list is used, q.push can be implemented by inserting the element at the front of the linked list, which is an $O(1)$ operation.

However, q.pop and q.top require scanning the entire list to find the maximum-priority element.

Q2. Priority Queue Using an Unsorted Array

How will we implement a priority queue over an unsorted array?

- The maximum element can be maintained at the end of the list.
- `q.pop` and `q.top` can then be implemented in $O(1)$.
- However, `q.push(e)` needs to scan the entire list to find the appropriate position to insert `e`, which is an $O(n)$ operation.

Heap

A priority queue is one of the fundamental containers. Many other algorithms assume access to efficient priority queues.

We define a data structure called a heap that provides an efficient implementation for a priority queue.

A heap T is a binary tree such that the following properties hold:

- Structural property: All levels are full except the last one, and the last level is filled from left to right.
- Heap property: For each non-root node n , $\text{key}(n) \leq \text{key}(\text{parent}(n))$.

An example of a heap is shown in the assignment material.

Q3. Heap Property

Show that nodes on a path from the root to a leaf have keys in non-increasing order.

Build Heap

Input: A binary tree T that has the structural property.

If the structural property holds, then T can be represented as an array.

Output: A heap over the elements of T .

Algorithm: BuildHeap(Heap T)

```
for  $i := T.\text{size} - 1$  down to 0 do  
    Heapify( $T, i$ )
```

Algorithm: Heapify(Heap T, i)

```
 $c := \text{IndexWithLargestKey}(T, i, \text{left}(i), \text{right}(i))$   
// Assume  $T[i] = -\infty$  if  $i \geq T.\text{size}$ .  
  
if  $c == i$  then  
    return  
  
Swap( $T, c, i$ )  
Heapify( $T, c$ )
```

Algorithm: DeleteMax(Heap T)

```
Swap(T, 0, T.size - 1)
T.size := T.size - 1
Heapify(T, 0)
return T[T.size]
```